

Apostila de Preparação para Programação **C**

Rafael Coelho

1 de setembro de 2026

Prefácio

A linguagem C ocupa um papel fundamental na formação de profissionais da Computação. Embora linguagens modernas ofereçam mecanismos que abstraem diversos detalhes da implementação, compreender como um programa é executado, como os dados são organizados na memória e como os recursos do computador são utilizados continua sendo uma habilidade indispensável para quem deseja construir uma base sólida em programação.

Esta apostila foi elaborada como material de apoio às monitorias da disciplina de Linguagem C. Seu propósito não é substituir as aulas ou os livros clássicos da área, mas complementar o processo de aprendizagem por meio de explicações objetivas, exemplos comentados, diagramas de memória, exercícios progressivos e observações sobre os erros mais frequentes encontrados pelos estudantes.

Ao longo dos capítulos, a linguagem será apresentada não apenas como uma ferramenta para escrever código, mas como um meio para compreender os fundamentos da programação e o funcionamento interno dos computadores. Sempre que possível, os conceitos serão relacionados a situações práticas e acompanhados de representações visuais que auxiliem na construção de um modelo mental consistente sobre a execução dos programas.

Espero que sirva como um bom roteiro de estudos para aqueles que desejam consolidar os conhecimentos adquiridos em sala de aula, preparar-se para avaliações e, principalmente, desenvolver uma compreensão mais profunda dos princípios que fundamentam o mundo maravilhoso da programação baixo nível.

Bom estudo!

Como utilizar esta apostila

Esta apostila foi organizada para funcionar como um roteiro de aprendizado. Em vez de apresentar apenas conceitos e exemplos, cada capítulo segue uma estrutura didática pensada para conduzir o estudante desde a motivação inicial até a aplicação prática do conteúdo.

Cada capítulo inicia com uma breve introdução, contextualizando a importância do tema e sua relação com os assuntos já estudados. Em seguida, o desenvolvimento apresenta os conceitos fundamentais, exemplos comentados e trechos de código que ilustram sua aplicação prática. Sempre que pertinente, caixas de observação, curiosidades e erros comuns destacam informações importantes, aplicações interessantes e equívocos frequentes entre estudantes, especialmente aqueles que estão migrando de linguagens de alto nível, como Python, para a linguagem C.

Ao final de cada capítulo, um resumo sintetiza os principais conceitos estudados, seguido por uma lista de objetivos de aprendizagem que funciona como um checklist para revisão. Por fim, são propostos três exercícios em ordem crescente de dificuldade: o primeiro busca consolidar os conceitos apresentados; o segundo incentiva sua aplicação na resolução de pequenos problemas; e o terceiro propõe um desafio ou reflexão que prepara o terreno para os capítulos seguintes.

Recomenda-se que o leitor acompanhe os exemplos, implemente os programas apresentados e tente resolver os exercícios antes de consultar soluções ou materiais complementares. A aprendizagem da programação ocorre principalmente por meio da prática, da experimentação e da análise dos próprios erros. Esta apostila foi concebida para servir como um guia ao longo desse processo, podendo ser utilizada tanto no estudo individual quanto como material de apoio às monitorias e às disciplinas introdutórias de programação em C.

Sumário

Prefácio	1
Como utilizar esta apostila	2
I Fundamentos de C	6
1 Introdução	7
1.1 O que significa programar?	8
1.2 Do problema ao algoritmo	8
1.3 Da linguagem humana à linguagem de máquina	9
1.4 O surgimento da linguagem C	10
1.5 Por que estudar C atualmente?	10
1.6 Estrutura de um programa em C	11
1.7 Compilando o primeiro programa	13
2 Variáveis, Tipos e Operadores	17
2.1 Variáveis e Declaração	17
2.2 Tipos de Dados Fundamentais	18
2.3 Representação dos Dados na Memória	19
2.4 Constantes e Modificadores	20
2.5 Operadores	20
2.5.1 Operadores Aritméticos	20
2.5.2 Operadores Relacionais	21
2.5.3 Operadores Lógicos	21

2.5.4	Operadores de Atribuição	21
2.5.5	Incremento e Decremento	21
2.6	Conversão de Tipos	22
2.7	Entrada e Saída de Dados	22
2.8	Escopo e Tempo de Vida	23
3	Controle de Fluxo	26
3.1	Condicionais	26
3.2	Laços	26
4	Funções	27
4.1	Declaração	27
4.2	Protótipos	27
4.3	Passagem por Valor	27
II	Organização dos Dados	28
5	Vetores	29
6	Strings	30
7	Arrays Multidimensionais	31
III	Gerenciamento de Memória	32
8	Ponteiros	33
9	Organização da Memória	34
9.1	Segmento de Código	34
9.2	Stack	34
9.3	Heap	34
9.4	Data	34
9.5	BSS	34

10 Alocação Dinâmica	35
 IV Programação Modular	 36
11 Structs e Enums	37
12 Modularização	38
12.1 Arquivos Header	38
12.2 Include Guards	38
12.3 Compilação Separada	38
12.4 Linkagem	38
 13 Estruturas de Dados Básicas	 39
 V Tópicos Avançados	 40
14 Manipulação Bit a Bit	41
15 Recursão	42
16 Depuração	43
 VI Apêndices	 44
A Perguntas Frequentes	45
B Erros Clássicos em C	46
C Questões Comentadas	47

Parte I

Fundamentos de C

1 Introdução

Programar é, antes de tudo, resolver problemas. Em sua essência, um programa de computador nada mais é do que uma sequência organizada de instruções capaz de transformar dados de entrada em resultados úteis. Porém, essa definição simples esconde um aspecto fundamental: computadores não compreendem intenções, ideias ou objetivos. Eles executam apenas instruções elementares, em uma ordem rigorosamente definida, realizando milhões ou até bilhões de operações por segundo.

Ao longo da história da Computação, diferentes linguagens de programação foram desenvolvidas para reduzir a distância entre a forma como os seres humanos pensam e a forma como os computadores executam instruções. Linguagens de alto nível, como Python, permitem que problemas complexos sejam descritos por meio de comandos simples e expressivos, abstraindo detalhes relacionados à organização da memória, ao gerenciamento de recursos e à arquitetura do processador. Essa capacidade torna o desenvolvimento de software mais produtivo e acessível, permitindo que o programador concentre seus esforços na lógica da solução.

Entretanto, compreender apenas essas abstrações é conhecer apenas uma parte do processo. Em diversas áreas da Computação, como sistemas operacionais, compiladores, sistemas embarcados, arquitetura de computadores, processamento de alto desempenho e desenvolvimento de drivers, torna-se necessário compreender como os dados são realmente armazenados, manipulados e transferidos pela máquina. É nesse contexto que a linguagem C assume um papel de destaque.

Desenvolvida no início da década de 1970 por Dennis Ritchie, nos Laboratórios Bell, a linguagem C surgiu com o objetivo de oferecer uma alternativa eficiente e portátil para o desenvolvimento do sistema operacional UNIX. Desde então, tornou-se uma das linguagens mais influentes da história da Computação, servindo de base para inúmeras outras linguagens modernas e permanecendo amplamente utilizada em aplicações nas quais desempenho, controle de memória e acesso direto ao *hardware* são requisitos fundamentais.

Estudar C não significa abandonar as facilidades oferecidas pelas linguagens mo-

dermas, mas compreender os mecanismos que tornam essas facilidades possíveis. Conceitos como ponteiros, organização da memória, pilha de execução, alocação dinâmica e representação binária dos dados constituem fundamentos presentes, de forma direta ou indireta, em praticamente todas as áreas da Computação. Assim, aprender C representa uma oportunidade de desenvolver uma compreensão mais profunda sobre o funcionamento dos programas e dos computadores.

Ao longo desta apostila, os conceitos serão apresentados de maneira gradual, sempre buscando estabelecer uma relação entre a escrita do código e o comportamento do programa durante sua execução. Sempre que possível, serão utilizados diagramas de memória, exemplos comentados e exercícios progressivos para auxiliar na construção de um modelo mental consistente sobre o funcionamento interno da linguagem.

1.1 O que significa programar?

Programar é, antes de tudo, uma forma de resolver problemas. Quando escrevemos um programa, estamos descrevendo, passo a passo, como um computador deve agir para transformar uma entrada em uma saída desejada. Não é sobre "escrever código" no sentido mecânico — é sobre pensar de forma estruturada, quebrar um problema grande em partes menores e traduzir esse raciocínio em instruções que uma máquina consiga executar sem ambiguidade.

Diferente de um ser humano, o computador não interpreta intenções. Ele não "entende" o que você quer dizer, apenas o que você efetivamente escreveu. Por isso, programar exige precisão: cada instrução precisa ser exata, cada condição precisa estar bem definida, cada passo precisa ter uma ordem clara. Essa exigência de rigor é, ao mesmo tempo, o que torna a programação desafiadora no início e extremamente poderosa depois que você domina o processo — porque uma vez que o computador sabe exatamente o que fazer, ele faz isso rapidamente, repetidamente e sem erros de execução.

Aprender a programar, portanto, é menos sobre decorar sintaxe e mais sobre desenvolver uma forma de pensar: o chamado **pensamento computacional**. É a habilidade de olhar para um problema do mundo real e enxergar nele uma sequência lógica de passos que pode ser automatizada.

1.2 Do problema ao algoritmo

Antes de qualquer linha de código, existe um problema. E antes de qualquer programa, existe um **algoritmo**: uma sequência finita e bem definida de passos que resolve esse

problema.

Pense em uma tarefa simples do dia a dia, como fazer um café. Existe uma ordem lógica de ações: ferver a água, moer o pó (se necessário), colocar o pó no filtro, despejar a água, esperar coar. Se você trocar a ordem dos passos ou pular algum, o resultado muda ou simplesmente não funciona. Um algoritmo de programação segue a mesma lógica, só que aplicado a problemas computacionais: somar dois números, ordenar uma lista de nomes, verificar se uma senha é válida, calcular a média de notas de uma turma.

O processo de desenvolvimento de software segue, de forma geral, este caminho:

1. **Entender o problema:** o que exatamente precisa ser resolvido?
2. **Projetar o algoritmo:** quais passos, em qual ordem, resolvem esse problema?
3. **Implementá-lo:** traduzir o algoritmo para uma linguagem de programação.
4. **Testar:** verificar se o programa realmente resolve o problema em diferentes situações.

É comum que iniciantes pulem direto para o passo 3, tentando escrever código antes de entender o problema ou pensar no algoritmo. Isso costuma gerar programas confusos e cheios de erros. Uma boa prática que vamos reforçar ao longo de toda a apostila é sempre rascunhar o algoritmo antes de programar, seja em português mesmo, seja em pseudocódigo, seja em um simples diagrama.

1.3 Da linguagem humana à linguagem de máquina

Nós pensamos e nos comunicamos em linguagem natural: português, inglês, e assim por diante. O computador, por outro lado, só entende sequências de 0s e 1s a chamada **linguagem de máquina**. Entre esses dois extremos existe uma escada de abstração, e é nela que as linguagens de programação se encaixam.

- No nível mais baixo está o **código binário**, que o processador executa diretamente.
- Um pouco acima está o **assembly**, que usa mnemônicos (como ‘MOV’, ‘ADD’, ‘JMP’) para representar instruções de máquina de forma um pouco mais legível para humanos, mas ainda extremamente ligada ao hardware.
- Acima disso estão as **linguagens de alto nível**, como C, Python, Java, entre outras, que usam palavras e estruturas mais próximas do raciocínio humano.

Quando você escreve um programa em C, esse código não é executado diretamente pelo processador. Ele precisa passar por um **compilador**, um programa que traduz o código-fonte escrito por você para linguagem de máquina, gerando um arquivo executável.

C ocupa uma posição interessante nessa escada: é considerada uma linguagem de **médio nível**. Ela tem estruturas legíveis para humanos (laços, condicionais, funções), mas também permite manipular a memória do computador de forma bastante direta algo que linguagens mais modernas costumam esconder do programador. É justamente esse equilíbrio que torna C tão importante de se entender, como veremos a seguir.

1.4 O surgimento da linguagem C

A linguagem C foi criada no início da década de 1970 por **Dennis Ritchie**, nos Laboratórios Bell (Bell Labs), com o objetivo original de desenvolver o sistema operacional **Unix**. Antes de C, sistemas operacionais eram escritos quase inteiramente em assembly um processo lento, difícil de manter e completamente dependente do hardware específico de cada máquina.

C nasceu como evolução de linguagens anteriores, como a **B** (criada por Ken Thompson) e a **BCPL**. A grande sacada de Ritchie foi criar uma linguagem poderosa o suficiente para escrever um sistema operacional inteiro, mas portátil o bastante para rodar em diferentes arquiteturas de hardware sem precisar reescrever tudo do zero.

O sucesso foi tão grande que, em pouco tempo, o próprio Unix foi reescrito em C, e a linguagem se espalhou junto com a adoção do sistema pelo mundo acadêmico e, depois, comercial. Em 1978, Brian Kernighan e Dennis Ritchie publicaram o livro *The C Programming Language*, obra que se tornou referência tão importante que muita gente ainda chama a versão clássica da linguagem de "C K&R" (Kernighan & Ritchie).

Com o tempo, C passou por processos de padronização formal ANSI C (1989), depois ISO C (C90, C99, C11, C17, C23) garantindo que compiladores diferentes, em sistemas diferentes, interpretassem a linguagem da mesma forma.

1.5 Por que estudar C atualmente?

Se você já programou em Python, provavelmente nunca precisou se preocupar com onde uma variável estava armazenada na memória. Em C, essa preocupação deixa de ser um detalhe e passa a fazer parte da programação. Então, mais de cinquenta anos depois de criada, por qual motivo C continua extremamente relevante? Não é apenas

por tradição ou nostalgia, existem razões práticas concretas:

- **C é a base de grande parte da computação moderna:** Sistemas operacionais, compiladores, bibliotecas e diversas linguagens de programação possuem componentes implementados em C. Compreender a linguagem significa compreender parte dos fundamentos sobre os quais a computação atual foi construída.
- **C permite compreender o funcionamento interno do computador:** Ao programar em C, passamos a lidar explicitamente com memória, ponteiros, organização dos dados e outros conceitos que normalmente permanecem ocultos em linguagens de alto nível. Essa experiência desenvolve uma compreensão mais profunda sobre como os programas realmente são executados.
- **C continua essencial em aplicações de baixo nível:** Sistemas embarcados, firmware, drivers de dispositivos, sistemas operacionais e diversas aplicações que exigem controle direto do hardware ainda utilizam C como uma das principais linguagens de desenvolvimento.

Em resumo: você não está aprendendo C apenas para "escrever programas em C". Está aprendendo os fundamentos que sustentam praticamente toda a computação moderna.

1.6 Estrutura de um programa em C

Tradicionalmente, o primeiro programa apresentado em uma linguagem de programação é aquele que exibe uma simples mensagem na tela. Embora pareça um exemplo modesto, esse programa reúne os principais elementos estruturais que aparecerão em praticamente todos os programas escritos em C. Por esse motivo, ele será utilizado como ponto de partida para compreender a organização básica de um programa.

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("Hello, \uworld!\n");
6     return 0;
7 }
```

Vamos analisar, a seguir, cada parte desse programa.

Curiosidade

Embora hoje seja tradição iniciar o estudo de uma linguagem de programação com um programa que imprime “Hello, World!”, essa expressão foi popularizada por Brian Kernighan. Ela apareceu inicialmente em um exemplo do livro *A Tutorial Introduction to the Language B* (1973) e, posteriormente, tornou-se mundialmente conhecida por meio do clássico *The C Programming Language* (1978), escrito por Brian Kernighan e Dennis Ritchie.

Usando a numeração das linhas temos em ordem crescente:

```
#include <stdio.h>
```

Essa linha é uma **diretiva de pré-processador**. Antes mesmo da compilação, ela instrui o pré-processador a incluir o conteúdo da biblioteca padrão de entrada e saída (`'stdio.h'`, de **standard input/output**), que contém a declaração de funções como `'printf'` (usada para exibir texto na tela) e `'scanf'` (usada para ler dados digitados pelo usuário).

```
int main(void)
```

Toda execução de um programa em C começa em uma função chamada `'main'`. É o ponto de entrada de todo programa em C. O `'int'` antes do nome indica que essa função devolve um número inteiro ao sistema operacional quando termina normalmente usado para indicar se o programa foi executado com sucesso ou não. O `'void'` entre parênteses indica que essa função não recebe parâmetros de entrada.

```
{ ... }
```

As chaves delimitam o corpo da função, isto é, o conjunto de instruções que será executado quando a função for chamada. No caso da função `main`, esse bloco corresponde ao código executado quando o programa é iniciado.

```
printf("Hello, world!\n");
```

A função `printf` é utilizada para exibir informações na tela. Nesse exemplo, ela recebe como argumento a sequência de caracteres `"Hello, world!\n"`, passando como argumento o texto que queremos exibir. O `'\n'` no final é um caractere especial que representa uma quebra de linha. Note o ponto e vírgula (`';'`) ao final em C, ele marca o fim de cada instrução, e sua ausência é um dos erros mais comuns entre iniciantes.

```
return 0;
```

Essa linha finaliza a função `'main'`, devolvendo o valor `'0'` ao sistema operacional. Por convenção, `'0'` significa que o programa terminou sem erros. Em programas simples, essa linha costuma aparecer no final da função `main`. Ao longo da apostila veremos que outras funções também podem utilizar a instrução `return` para devolver resultados

ao programa.

De forma geral, todo programa em C segue este esqueleto:

```
1 #include <biblioteca_1>
2 #include <biblioteca_2>
3 ...
4
5 int main(void) {
6     // instruções do programa
7     return 0;
8 }
```

Naturalmente, programas reais podem conter diversas outras funções, declarações, estruturas de dados e bibliotecas. Entretanto, a estrutura apresentada acima representa o núcleo de um programa em C e servirá como base para os próximos capítulos.

Observação

Neste momento, não se preocupe em memorizar todos os detalhes do programa. Ao longo dos próximos capítulos, cada um desses elementos será estudado individualmente. O objetivo desta seção é apenas apresentar a estrutura geral de um programa em C e familiarizar o leitor com sua organização.

1.7 Compilando o primeiro programa

Diferente de linguagens interpretadas, um programa em C precisa ser compilado antes de ser executado. O compilador lê o código-fonte (o arquivo ‘c’ que você escreveu) e gera um arquivo executável, específico para o sistema operacional e a arquitetura de hardware em que foi compilado.

Um dos compiladores mais usados, especialmente em ambientes Linux e no ensino, é o **GCC** (GNU Compiler Collection). Supondo que você tenha salvo o programa da seção 1.6 em um arquivo chamado ‘ola_mundo.c’, o processo básico de compilação e execução no terminal seria:

```
1 aluno@ubuntu:~$ gcc ola_mundo.c -o ola_mundo
2
3 aluno@ubuntu:~$ ./ola_mundo
```

Explicando cada parte:

<code>gcc</code>	Invoca o compilador GCC.
<code>ola_mundo.c</code>	Arquivo-fonte que será compilado.
<code>-o ola_mundo</code>	Define o nome do executável gerado.
<code>./ola_mundo</code>	Executa o programa localizado no diretório atual.

Assim, se tudo estiver correto, a saída no terminal será:

```
1 Hello, world!
```

Caso exista algum erro de sintaxe como esquecer um ponto e vírgula ou uma chave o compilador vai recusar a compilação e exibir mensagens de erro apontando a linha e o tipo de problema encontrado. Aprender a ler essas mensagens de erro é uma habilidade tão importante quanto aprender a escrever o código em si, e vamos treinar isso bastante ao longo da apostila.

Resumo do Capítulo

Parabéns! Você acabou de escrever, compilar e executar seu primeiro programa em C. Embora simples, esse exemplo percorreu todas as etapas fundamentais do desenvolvimento: partiu de um arquivo de código-fonte, passou pelo processo de compilação e resultou em um programa executável.

Ao longo do capítulo, você também conheceu o contexto histórico da linguagem C e refletiu sobre o verdadeiro papel da programação: utilizar o computador como ferramenta para resolver problemas por meio de algoritmos.

Nos próximos capítulos, ampliaremos gradualmente essa estrutura, introduzindo variáveis, operadores, estruturas de controle e funções, até que você seja capaz de desenvolver programas cada vez mais completos.

Ao final deste capítulo, você deve ser capaz de:

- Explicar, em linhas gerais, como surgiu a linguagem C e sua importância histórica.
- Diferenciar algoritmo de programa.
- Identificar os principais componentes da estrutura de um programa em C.
- Compreender o papel das diretivas de pré-processador e da função `main`.
- Compilar e executar um programa simples utilizando o GCC.
- Interpretar mensagens básicas de erro emitidas pelo compilador.

Erros comuns

Erro comum

Esquecer o ponto e vírgula (;)

Para quem teve o primeiro contato com programação em Python, é recorrente esquecer o ponto e vírgula ao final das instruções. Em C, praticamente toda instrução deve ser finalizada com `;`. Quando isso não acontece, o compilador informa um erro de sintaxe.

Python

```
1 print("Betas usam Python.")
```

C

```
1 printf("Trivial.");
```

Exercícios

Exercício

Fixação

Explique, com suas palavras, por que a linguagem C continua sendo importante para a formação de um profissional de Computação, mesmo existindo linguagens

mais modernas.

Aplicação

Escreva um programa em C que exiba seu nome, idade e cidade que nasceu na tela (com a exata formatação a seguir):

```
1 Rafael, 21
2 Aracaju-SE.
```

Em seguida, compile e execute o programa utilizando o GCC.

Reflexão

Neste capítulo, vimos que um programa em C precisa ser compilado antes de ser executado. Em Python, normalmente basta executar o arquivo diretamente.

Pesquise, em poucas linhas, qual é a diferença entre uma linguagem compilada e uma linguagem interpretada. Em sua opinião, quais vantagens e desvantagens cada abordagem apresenta?

2 Variáveis, Tipos e Operadores

Todo programa existe para representar informações e operar sobre elas. Se você já teve contato com Python, provavelmente está acostumado a pensar em uma variável simplesmente como "uma caixa que guarda um valor" você escreve `idade = 20` e pronto, o Python descobre sozinho que aquilo é um número inteiro.

Em C, esse raciocínio precisa ser mais preciso. Antes de guardar qualquer valor, precisamos responder três perguntas:

- Qual é o tipo desse dado?
- Quanto espaço ele ocupa na memória?
- Como esse valor é efetivamente representado, em bits, dentro desse espaço?

Essas perguntas podem parecer burocráticas no início, mas são exatamente o que torna C uma linguagem tão próxima do funcionamento real do computador. Neste capítulo, vamos construir a base para pensar em variáveis não como "caixas mágicas", mas como regiões de memória com um tipo bem definido.

2.1 Variáveis e Declaração

Uma variável é um espaço nomeado na memória do computador, usado para armazenar um valor que pode ser lido e modificado durante a execução do programa. Em C, toda variável precisa ser **declarada** antes de ser usada ou seja, você precisa informar ao compilador qual é o tipo dela e qual nome ela vai ter.

```
1 int idade;  
2 idade = 20;  
3  
4 int ano = 2026;
```

Note que existem duas ações diferentes acontecendo:

- **Declaração:** `int idade;` aqui você está dizendo ao compilador "reserve espaço na memória para um número inteiro chamado idade".
- **Atribuição:** `idade = 20;` aqui você está colocando o valor 20 nesse espaço já reservado.

Quando as duas coisas acontecem na mesma linha, como em `int ano = 2026;`, chamamos isso de inicialização declaração e atribuição em um único passo.

Regras para identificadores (nomes de variáveis) em C:

1. Podem conter letras, dígitos e underscore (`_`).
2. Não podem começar com um dígito.
3. São case-sensitive (`idade` e `Idade` são variáveis diferentes).
4. Não podem ser palavras reservadas da linguagem (como `int`, `return`, `if`).

A diferença em relação ao Python: em Python, uma mesma variável pode receber valores de tipos diferentes ao longo do programa (`x = 10` e depois `x = "texto"` são ambos válidos). Em C isso não é possível uma vez declarada com um tipo, a variável só pode armazenar valores compatíveis com aquele tipo durante toda a sua existência.

Erro comum

Declarar uma variável local e tentar usá-la antes de atribuir um valor:

```
1 int total;  
2 printf("%d", total); // comportamento indefinido!
```

Diferente de variáveis globais (que começam zeradas automaticamente), variáveis locais não têm um valor inicial garantido. Elas contêm "lixo de memória" o que quer que estivesse naquele espaço antes.

Sempre inicialize suas variáveis antes de usá-las.

2.2 Tipos de Dados Fundamentais

C possui alguns tipos básicos, chamados de **tipos primitivos**, que servem de base para tudo o que vamos construir daqui em diante:

Tipo	Representa	Exemplo
char	um único caractere	char letra = 'A';
int	um número inteiro	int quantidade = 42;
float	número real (precisão simples)	float altura = 1.75f;
double	número real (precisão dupla)	double pi = 3.14159265;
void	ausência de tipo/valor	usado principalmente em funções

Mais adiante, vamos discutir sobre o tipo `bool`, que representa valores verdadeiro/falso, em C com mais calma quando chegarmos às estruturas condicionais.

Por enquanto, não vamos nos aprofundar em modificadores como `short`, `long`, `signed` e `unsigned` eles merecem atenção própria e voltaremos a eles na Seção 2.4.

A ideia central desta seção é a seguinte:

Em C, o tipo faz parte da definição da variável e determina como seus bits devem ser interpretados.

Ou seja: os mesmos 4 bytes na memória podem representar um número inteiro ou um número em ponto flutuante, dependendo exclusivamente do tipo declarado. O tipo não é um detalhe burocrático é o que dá significado aos bits armazenados.

2.3 Representação dos Dados na Memória

Este é um dos pontos em que C se diferencia claramente de linguagens de mais alto nível: podemos e devemos pensar em como os dados são fisicamente armazenados.

Considere a declaração:

```
1 int x = 10;
```

Na maioria dos sistemas atuais, um `int` ocupa **4 bytes** (32 bits) de memória. O valor 10, representado em binário, fica armazenado assim:

00000000	00000000	00000000	00001010
----------	----------	----------	----------

4 bytes

Não vamos entrar em detalhes sobre representação de números negativos (complemento de dois) neste momento isso terá seu espaço mais à frente. O que importa agora é fixar uma cadeia de raciocínio que vai acompanhar todo o restante da apostila:

variável → tipo → bytes → memória.

Toda variável que você declara reserva um espaço de memória cujo tamanho é determinado pelo seu tipo. Essa ideia é a base sobre a qual construiremos, mais adiante, os capítulos de ponteiros e de *structs* então vale a pena parar um instante e garantir que essa imagem mental ficou clara antes de avançar.

2.4 Constantes e Modificadores

Às vezes queremos declarar uma variável cujo valor não deve mudar ao longo do programa. Para isso, usamos a palavra-chave **const**:

```
1 const int idade = 20;
```

Qualquer tentativa de modificar `idade` depois disso resultará em erro de compilação o que é útil para proteger valores que representam configurações fixas, limites ou constantes matemáticas.

C também oferece **modificadores de tipo**, que ajustam o tamanho ou a forma como um número é interpretado:

```
1 unsigned int // apenas valores não negativos
2 long int     // inteiro com mais bits (maior faixa de valores)
3 short int    // inteiro com menos bits (faixa menor)
```

Não vamos nos aprofundar nesses modificadores agora pois eles terão uma seção própria mais adiante, quando discutirmos faixas de valores e *overflow* com mais detalhe.

2.5 Operadores

Operadores são símbolos que indicam ao compilador que operação deve ser realizada sobre um ou mais valores (chamados de *operandos*).

2.5.1 Operadores Aritméticos

```
1 + soma
2 - subtracao
3 * multiplicacao
4 / divisao
5 % modulo (resto da divisao inteira)
```

2.5.2 Operadores Relacionais

Usados para comparar dois valores, sempre resultando em verdadeiro (1) ou falso (0):

```
1 <      menor que
2 >      maior que
3 <=     menor ou igual
4 >=     maior ou igual
5 ==     igual a
6 !=     diferente de
```

2.5.3 Operadores Lógicos

Usados para combinar expressões relacionais:

```
1 &&     e (AND)
2 ||     ou (OR)
3 !      negacao (NOT)
```

2.5.4 Operadores de Atribuição

```
1 =      atribuicao simples
2 +=     soma e atribui
3 -=     subtrai e atribui
4 *=     multiplica e atribui
5 /=     divide e atribui
6 %=     calcula o modulo e atribui
```

Por exemplo, `total += 5;` é equivalente a `total = total + 5;`.

2.5.5 Incremento e Decremento

```
1 ++     incremento
2 --     decremento
```

```
1 i++;
2 ++i;
```

Vale destacar algo que costuma passar despercebido: **++ é um operador, e não simplesmente uma abreviação textual de `i = i + 1`**. Essa distinção parece sutil

agora, mas vai se tornar muito relevante quando estudarmos ponteiros e a diferença entre as formas pré-fixada (`++i`) e pós-fixada (`i++`) em expressões mais complexas, como `vetor[i++]`.

2.6 Conversão de Tipos

Em C, é possível e às vezes necessário converter um valor de um tipo para outro. Essa conversão é chamada de *casting*.

```
1 int a = 5;
2 int b = 2;
3
4 double resultado = (double) a / b;
```

Aqui está uma diferença importante em relação ao Python: em Python, `5 / 2` já retorna `2.5` automaticamente. Em C, `a / b` sendo ambos `int` realiza uma **divisão inteira**, descartando a parte decimal e resultando em `2`. Para obter o resultado com casas decimais, é preciso converter explicitamente um dos operandos para `double` (ou `float`) antes da divisão, como feito no exemplo acima.

Conversões também acontecem de forma implícita, quando atribuímos um valor de um tipo a uma variável de outro tipo:

```
1 int x = 10;
2 double y = x;           // conversao implicita: int -> double (sem
                           perda de dados)
```

```
1 double x = 10.5;
2 int y = (int) x;        // conversao explicita: double -> int (
                           perde a parte decimal, y = 10)
```

De forma geral, converter de um tipo "menor" para um "maior" (como `int` para `double`) é seguro. O caminho inverso pode causar perda de informação, e por isso é uma boa prática torná-lo explícito com um *cast*, como em `(int) x`.

2.7 Entrada e Saída de Dados

No Capítulo 1, você já usou a função `printf` para exibir uma mensagem fixa na tela. Agora que conhecemos variáveis e tipos, podemos usá-la e sua "parceira" `scanf` de forma sistemática, para exibir e ler valores dinâmicos.

```
1 printf("%d", idade);  
2 scanf("%d", &idade);
```

Cada tipo de dado tem um **especificador de formato** correspondente, usado tanto em `printf` quanto em `scanf`:

```
1 %d    int  
2 %f    float  
3 %lf   double  
4 %c    char  
5 %s    string (vetor de char)
```

Erro comum

```
1 scanf("%d", idade);    // errado  
2 scanf("%d", &idade);   // correto
```

O `scanf` precisa saber **onde**, na memória, deve escrever o valor lido e não apenas qual valor a variável tem no momento. Por isso, é necessário passar o **endereço de memória** da variável, usando o operador `&` antes do nome dela. Omitir o `&` é um dos erros mais comuns entre quem está começando em C, e o compilador nem sempre vai te avisar com clareza sobre isso.

Isso naturalmente levanta uma pergunta:

"Por que preciso colocar `&`?"

A resposta completa para essa pergunta pertence ao capítulo sobre ponteiros, mais adiante. Por ora, fica o registro consciente de que existe algo acontecendo por trás dessa sintaxe que ainda vamos explorar em profundidade. É importante não esconder isso do aluno, mas também não se aprofundar antes da hora.

2.8 Escopo e Tempo de Vida

Toda variável tem um **escopo** a região do código onde ela é visível e pode ser usada e um **tempo de vida** o período durante o qual ela existe na memória.

- **Escopo local:** variáveis declaradas dentro de uma função (ou de um bloco delimitado por chaves) só existem e só podem ser acessadas dentro daquele bloco.

- **Escopo global:** variáveis declaradas fora de qualquer função, no topo do arquivo, podem ser acessadas por qualquer função do programa.

```
1 int contador_global = 0; // escopo global
2
3 int main(void) {
4     // escopo local, existe apenas dentro de main
5     int idade_local = 20;
6     return 0;
7 }
```

De forma geral, variáveis locais existem apenas enquanto a função em que foram declaradas está em execução assim que a função termina, esse espaço de memória é liberado. Variáveis globais, por outro lado, existem durante toda a execução do programa.

Resumo do Capítulo

Neste capítulo, deixamos de tratar variáveis como simples "caixas de valores" e passamos a enxergá-las como regiões de memória com tipo, tamanho e representação bem definidos. Vimos os tipos fundamentais de C, como declarar e inicializar variáveis, os principais operadores da linguagem, como converter entre tipos e como realizar entrada e saída básica de dados. Também demos os primeiros passos sobre escopo um assunto que vai se tornar cada vez mais importante conforme nossos programas crescerem.

Ao final deste capítulo, você deve ser capaz de:

- Declarar e inicializar variáveis em C.
- Identificar os principais tipos fundamentais.
- Explicar a relação entre tipos e representação dos dados na memória.
- Utilizar operadores aritméticos, relacionais e lógicos.
- Realizar conversões de tipos.
- Utilizar `printf` e `scanf` para entrada e saída simples.
- Diferenciar escopo local e global.

Exercícios

Exercício

Fixação

Declare variáveis de tipos `int`, `float` e `char` para representar, respectivamente, a idade, a altura (em metros) e a inicial do primeiro nome de uma pessoa. Inicialize cada uma com um valor de sua escolha e utilize `printf` para exibir os três valores na tela, cada um com o especificador de formato adequado.

Aplicação

Escreva um programa que leia, via `scanf`, dois números inteiros digitados pelo usuário. O programa deve calcular e exibir a soma, a diferença, o produto e o resto da divisão entre eles.

Desafio

Analise o programa a seguir:

```
1 int a = 5;  
2 int b = 2;  
3  
4 printf("%d\n", a / b);  
5 printf("%f\n", (double)a / b);
```

Execute mentalmente (ou compile e rode, se preferir) esse trecho e responda: **por que as duas operações produzem resultados diferentes?**

Tente formular sua resposta usando os conceitos de tipo e representação de dados discutidos neste capítulo e não apenas como uma regra decorada sobre operadores.

Material Complementar

Neste capítulo foram utilizados diversos exemplos práticos. As implementações completas, listas adicionais de exercícios, slides da monitoria e perguntas frequentes encontram-se disponíveis no repositório da disciplina.

3 Controle de Fluxo

Até agora, todos os nossos programas seguiram um caminho único: a execução começa na primeira linha de `main` e segue, sequencialmente, até a última. Isso é suficiente para calcular expressões e exibir resultados, mas está longe de representar a maioria dos problemas reais. Precisamos que o computador seja capaz de **tomar decisões** ("se a senha estiver correta, libere o acesso") e de **repetir ações** ("some cada nota digitada até o usuário informar que terminou").

É exatamente isso que as estruturas de **controle de fluxo** nos permitem fazer: alterar a ordem natural de execução das instruções, com base em condições que só podem ser avaliadas durante a execução do programa. Neste capítulo, vamos estudar as duas grandes famílias dessas estruturas em C: as **condicionais**, que desviam o fluxo, e os **laços** (ou *loops*), que o repetem.

3.1 Condicionais

Estruturas condicionais permitem que um trecho de código seja executado apenas quando uma determinada condição for verdadeira. Em C, uma condição é sempre uma expressão que resulta em um valor: 0 é considerado falso, e qualquer valor diferente de zero é considerado verdadeiro.

3.1.1 A estrutura `if`

A forma mais simples de decisão é o `if`:

```
1 int idade = 18;
2
3 if (idade >= 18) {
4     printf("Maior de idade\n");
5 }
```

O bloco entre chaves só é executado se a condição dentro dos parênteses for verdadeira. Se a condição for falsa, o bloco é simplesmente ignorado e a execução continua na linha seguinte ao `if`.

3.1.2 if-else

Quando queremos executar um bloco alternativo caso a condição seja falsa, usamos `else`:

```
1 int idade = 15;
2
3 if (idade >= 18) {
4     printf("Maior de idade\n");
5 } else {
6     printf("Menor de idade\n");
7 }
```

3.1.3 Encadeando condições com else if

Quando existem múltiplas condições mutuamente exclusivas, podemos encadeá-las:

```
1 int nota = 7;
2
3 if (nota >= 9) {
4     printf("Conceito A\n");
5 } else if (nota >= 7) {
6     printf("Conceito B\n");
7 } else if (nota >= 5) {
8     printf("Conceito C\n");
9 } else {
10     printf("Reprovado\n");
11 }
```

É importante notar que as condições são avaliadas **em ordem**, de cima para baixo, e a execução para no primeiro bloco cuja condição for verdadeira. Trocar a ordem das comparações acima (por exemplo, testar `nota >= 5` antes de `nota >= 7`) mudaria completamente o comportamento do programa.

Erro comum

Confundir o operador de atribuição (=) com o operador de igualdade (==) dentro de uma condição:

```
1 int senha = 1234;
2
3 if (senha = 0) {    // errado: atribui 0 a senha, e a
4     condicao vira falsa
5     printf("Acesso negado\n");
6 }
7
8 if (senha == 0) {   // correto: compara senha com 0
9     printf("Acesso negado\n");
10 }
```

O trecho `senha = 0` não é uma comparação é uma atribuição, que **sobrescreve** o valor de `senha` e ainda avalia como falso (porque o valor atribuído foi 0). Esse é um dos erros mais traiçoeiros em C, porque o compilador geralmente não o considera um erro de sintaxe, apenas um possível aviso (*warning*). **Sempre revise suas condições com atenção redobrada.**

3.1.4 Aninhamento e chaves opcionais

Blocos de `if` podem conter outros `if` dentro deles, formando condicionais aninhadas:

```
1 if (idade >= 18) {
2     if (possui_documento) {
3         printf("Acesso liberado\n");
4     } else {
5         printf("Apresente um documento\n");
6     }
7 } else {
8     printf("Acesso negado: menor de idade\n");
9 }
```

Quando o bloco do `if` (ou do `else`) contém apenas uma única instrução, as chaves são tecnicamente opcionais:

```
1 if (idade >= 18)
2     printf("Maior de idade\n");
```

Embora válido, **recomendamos fortemente sempre usar chaves**, mesmo em blocos de uma linha só. Isso evita um erro clássico: se alguém adicionar uma segunda linha ao bloco no futuro, sem perceber que faltam chaves, apenas a primeira linha continuará condicionada ao `if` as demais serão executadas incondicionalmente, o que pode gerar bugs difíceis de detectar.

3.1.5 O operador ternário

Para condições simples que resultam na atribuição de um valor, C oferece uma forma compacta: o **operador ternário** `?`:

```
1 int idade = 20;
2 char *status = (idade >= 18) ? "maior" : "menor";
```

A expressão `condicao ? valor_se_verdadeiro : valor_se_falso` é equivalente a um `if-else` que atribui um valor a uma variável, mas escrita em uma única linha. É útil para casos simples, mas deve ser evitada quando a lógica se torna muito complexa nesse caso, um `if-else` tradicional costuma ser mais legível.

3.1.6 A estrutura switch

Quando uma mesma variável precisa ser comparada com vários valores possíveis, o `switch` pode ser uma alternativa mais organizada do que uma longa cadeia de `else if`:

```
1 int dia = 3;
2
3 switch (dia) {
4     case 1:
5         printf("Domingo\n");
6         break;
7     case 2:
8         printf("Segunda-feira\n");
9         break;
10    case 3:
11        printf("Terça-feira\n");
12        break;
13    default:
14        printf("Dia inválido\n");
15 }
```

O `switch` compara o valor da expressão (`dia`) com cada `case`, executando o bloco correspondente. O `default` é executado quando nenhum `case` corresponde ao valor de forma semelhante a um `else final`.

Erro comum

Esquecer o `break` ao final de cada `case`:

```
1 switch (dia) {  
2     case 1:  
3         printf("Domingo\n");  
4     case 2:  
5         printf("Segunda-feira\n"); // sera executado mesmo  
6         se dia == 1!  
7         break;  
8 }
```

Sem o `break`, a execução de um `switch` **continua** para o próximo `case`, executando-o também, independentemente da condição um comportamento chamado de *fall-through*. Às vezes esse comportamento é usado intencionalmente, mas na grande maioria dos casos ele é um erro de digitação com consequências sérias. Revise sempre se cada `case` termina com `break` (ou `return`, quando dentro de uma função).

O `switch` em C só funciona com valores de tipos inteiros (incluindo `char`) e não aceita, por exemplo, comparações com `strings` ou intervalos de valores. Limitações importantes a se ter em mente ao decidir entre um `switch` e uma cadeia de `if-else`.

3.2 Laços

Laços (também chamados de *loops*) permitem repetir um bloco de instruções enquanto uma condição for satisfeita, evitando que precisemos repetir manualmente o mesmo código várias vezes.

3.2.1 O laço while

O `while` repete um bloco de código **enquanto** sua condição for verdadeira. A condição é testada **antes** de cada repetição:

```
1 int contador = 1;  
2
```

```
3 while (contador <= 5) {  
4     printf("%d\n", contador);  
5     contador++;  
6 }
```

Se a condição for falsa já na primeira verificação, o bloco do **while** nunca é executado.

3.2.2 O laço do-while

O **do-while** é semelhante ao **while**, mas testa a condição **depois** de executar o bloco o que garante que o bloco seja executado pelo menos uma vez:

```
1 int opcao;  
2  
3 do {  
4     printf("Digite 0 para sair: ");  
5     scanf("%d", &opcao);  
6 } while (opcao != 0);
```

Essa estrutura é particularmente útil em situações como menus interativos, em que queremos executar a ação (exibir o menu) antes mesmo de sabermos se o usuário quer continuar ou não.

3.2.3 O laço for

O **for** é a estrutura de repetição mais utilizada quando sabemos, de antemão, quantas vezes (ou sob que condição controlada) o laço deve se repetir. Ele reúne três partes em uma única linha: inicialização, condição e incremento (ou decremento).

```
1 for (int i = 1; i <= 5; i++) {  
2     printf("%d\n", i);  
3 }
```

As três partes do **for** são executadas na seguinte ordem lógica:

1. A **inicialização** (`int i = 1`) é executada uma única vez, no início.
2. A **condição** (`i <= 5`) é testada antes de cada repetição. Se for falsa, o laço termina.
3. O bloco de código é executado.

4. O **incremento** (`i++`) é executado ao final de cada repetição, e o processo volta ao passo 2.

O `for` e o `while` são, no fundo, formas equivalentes de expressar a mesma ideia o exemplo acima poderia ser reescrito com um `while`:

```
1 int i = 1;
2 while (i <= 5) {
3     printf("%d\n", i);
4     i++;
5 }
```

Na prática, usamos `for` quando o número de repetições (ou o critério de parada) é conhecido de antemão, e `while` quando a repetição depende de uma condição que só se revela durante a execução do programa (como esperar por uma entrada específica do usuário).

Erro comum

Criar um laço infinito por esquecer de atualizar a variável de controle:

```
1 int i = 1;
2 while (i <= 5) {
3     printf("%d\n", i);
4     // faltou i++; o laço nunca termina!
5 }
```

Todo laço controlado por condição precisa de algo que, eventualmente, torne essa condição falsa. Antes de compilar um laço `while` ou `do-while`, pergunte-se sempre: **"o que, dentro deste bloco, está caminhando para tornar a condição falsa?"** Se a resposta for "nada", você encontrou um laço infinito antes mesmo de executá-lo.

3.2.4 `break` e `continue`

Duas palavras-chave permitem alterar o comportamento padrão de um laço:

- **`break`** interrompe o laço imediatamente, saindo dele por completo.
- **`continue`** interrompe apenas a repetição atual, pulando direto para a próxima verificação da condição.

```
1 for (int i = 1; i <= 10; i++) {
2     if (i == 5) {
3         break; // encerra o laço quando i for 5
4     }
5     printf("%d\n", i);
6 }
```

```
1 for (int i = 1; i <= 10; i++) {
2     if (i % 2 == 0) {
3         continue; // pula os numeros pares
4     }
5     printf("%d\n", i);
6 }
```

3.2.5 Laços aninhados

Assim como as condicionais, laços também podem ser aninhados — um laço dentro de outro. Isso é especialmente útil para percorrer estruturas bidimensionais, como veremos mais adiante ao estudarmos matrizes:

```
1 for (int i = 1; i <= 3; i++) {
2     for (int j = 1; j <= 3; j++) {
3         printf("(%d, %d) ", i, j);
4     }
5     printf("\n");
6 }
```

Note que, a cada repetição do laço externo (controlado por `i`), o laço interno (controlado por `j`) é executado **por completo** antes que o laço externo avance para a próxima repetição. Entender essa ordem de execução é fundamental para trabalhar corretamente com laços aninhados.

Resumo do Capítulo

Neste capítulo, aprendemos a alterar o fluxo sequencial padrão de um programa em C. Vimos como tomar decisões com `if`, `else if`, `else` e `switch`, e como repetir blocos de código com `while`, `do-while` e `for`. Também vimos como controlar o comportamento de um laço com `break` e `continue`, e discutimos armadilhas comuns, como confundir =

com `==` e esquecer de atualizar a condição de um laço. Essas estruturas são a espinha dorsal de praticamente todo programa não trivial, e vamos utilizá-las constantemente dos próximos capítulos em diante.

Ao final deste capítulo, você deve ser capaz de:

- Utilizar `if`, `else if` e `else` para tomar decisões.
- Utilizar o operador ternário em condições simples.
- Utilizar `switch` para comparar uma variável com múltiplos valores possíveis.
- Diferenciar `while`, `do-while` e `for`, e escolher a estrutura mais adequada a cada situação.
- Utilizar `break` e `continue` para controlar a execução de um laço.
- Identificar e evitar laços infinitos.
- Trabalhar com condicionais e laços aninhados.

Exercícios

Exercício 1 — Fixação

Escreva um programa que leia um número inteiro digitado pelo usuário e informe, usando `if-else`, se ele é positivo, negativo ou igual a zero.

Exercício 2 — Aplicação

Escreva um programa que utilize um laço `for` para exibir todos os números de 1 a 100 que sejam múltiplos de 3 **ou** de 5. Em seguida, utilizando um segundo laço, calcule e exiba a soma de todos esses números.

Exercício 3 — Desafio

Escreva um programa que simule um menu simples utilizando `do-while` e `switch`. O menu deve oferecer três opções (por exemplo, "1. Somar dois números", "2. Verificar se um número é par ou ímpar", "0. Sair") e continuar sendo exibido até que o usuário escolha a opção de sair.

Depois de implementar o programa, reflita: por que o **do-while** é mais adequado do que um **while** comum nesse cenário? O que mudaria no comportamento do programa se você tivesse usado um **while** no lugar?

4 Funções

4.1 Declaração

4.2 Protótipos

4.3 Passagem por Valor

Parte II

Organização dos Dados

5 Vetores

6 Strings

7 Arrays Multidimensionaux

Parte III

Gerenciamento de Memória

8 Ponteiros

```
1 int x = 10;  
2 int *p = &x;
```

Importante

Nunca utilize um ponteiro sem inicializá-lo.

9 Organização da Memória

9.1 Segmento de Código

9.2 Stack

9.3 Heap

9.4 Data

9.5 BSS

10 Alocação Dinâmica

Exercícios

Parte IV

Programação Modular

11 Structs e Enums

12 Modularização

12.1 Arquivos Header

12.2 Include Guards

12.3 Compilação Separada

12.4 Linkagem

13 Estruturas de Dados Básicas

Parte V

Tópicos Avançados

14 Manipulação Bit a Bit

15 Recursão

16 Depuração

Parte VI

Apêndices

A Perguntas Frequentes

B Erros Clássicos em C

C Questões Comentadas